



Advanced Numerical Methods
Final Project
Avi Kutai
Nave Markovich

Part 1

I. Q1

The 2D steady-state heat conduction equation $\nabla \cdot (k \nabla T) = 0$ is solved using a Finite Volume Method (FVM). In the Finite Volume formulation of the heat equation, the thermal conductivity k must be evaluated at the faces of the control volumes (e.g., face e between node P and neighbour E) to calculate the diffusive flux. Given the large discontinuity between the conductivity values $k_1 = 10^{-3}$ and $k_2 = 100$, using a simple Arithmetic Mean ($k_e \approx (k_P + k_E) / 2$) is physically incorrect. An arithmetic mean would skew the effective conductivity towards the larger value, artificially increasing heat transfer across the interface and failing to represent the insulating property of the lower conductivity material. Instead, we employ the Harmonic Mean to calculate the interface conductivity. This method is derived from the requirement of heat flux continuity across the interface. The thermal resistance between two cell centers is treated as two resistors in series. Just as the total resistance of a series circuit is the sum of individual resistances, the effective conductivity k_{face} is dominated by the more resistive (lower conductivity) material.

The effective conductivity at the interface is calculated as:

$$k_{face} = \frac{2k_P k_E}{k_P + k_E}$$

This ensures that if either k_P or k_E approaches zero (insulator), the interface conductivity correctly approaches zero, preserving the physical integrity of the solution near the material boundaries defined in the problem geometry.

The finite volume formulation was applied by integrating the steady heat equation over each cell and converting the divergence term into a balance of diffusive heat fluxes through the cell faces. On the resulting cell-centered grid, each face contribution leads to coefficients of order k / h^2 . At the left and right Dirichlet boundaries, the first and

last unknowns are only $h/2$ away from the wall, so the boundary contribution appears with an effective coefficient $2k/h^2$. The top and bottom boundaries were treated as zero-flux Neumann conditions, and the final algebraic system was solved using Gauss-Seidel by-lines with sweeps in the x -direction, where each vertical line is solved as a tridiagonal system using the Thomas algorithm. This treatment was important for keeping both the finite-volume discretization and the boundary implementation consistent in the presence of the strong conductivity jump.

The initial temperature field was chosen as a linear interpolation between the left and right boundary values in order to improve convergence. Convergence was declared when the infinity norm of the difference between two successive iterates dropped below 10^{-6} .

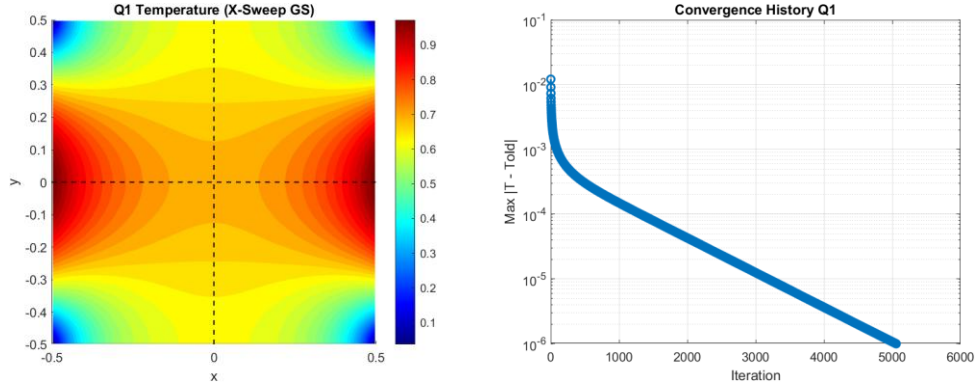


Figure 1: Q1 results

The results of Question 1 show that the finite-volume formulation, together with harmonic averaging and the line Gauss-Seidel solver, produces a stable and physically consistent steady-state solution for a problem with a very strong conductivity jump. The temperature field follows the imposed boundary conditions and shows the expected change in thermal behavior across the material interface: heat is conducted much more efficiently in the high-conductivity region, while stronger thermal resistance is observed in the low-conductivity region. The convergence history also indicates that the line-by-line iterative procedure is robust for this problem and is able to reach the prescribed tolerance without numerical instability.

II. Q2

For Question 2, we implemented a Geometric Multigrid V-cycle using a cell-centered Finite Volume discretization. The sharp jump in thermal conductivity was

handled using harmonic averaging at the cell faces, exactly as we did in Question 1. However, this extreme material interface directly influenced our choice of Multigrid transfer operators. To prevent physically smearing the boundary between the two materials on coarser grids, we avoided bilinear interpolation. Instead, we specifically chose a 4-point conservative average for restriction and piecewise constant expansion for prolongation. This approach prevents artificial conductivity blending and ensures the energy balance is strictly preserved across all grid levels.

Runtime efficiency was another major focus, especially keeping in mind the complexity requirements for Question 4. To maintain the theoretical $O(N)$ performance of Multigrid, we minimized the use of nested for loops. We fully vectorized the residual computations and used MATLAB's built-in `repelem` function to handle the prolongation step efficiently. Furthermore, we optimized the V-cycle architecture by assembling the coefficient matrix exactly once per grid level, rather than redundantly recalculating the physics during every single sweep of the line Gauss-Seidel smoother. This drastically reduced our overall computation time.

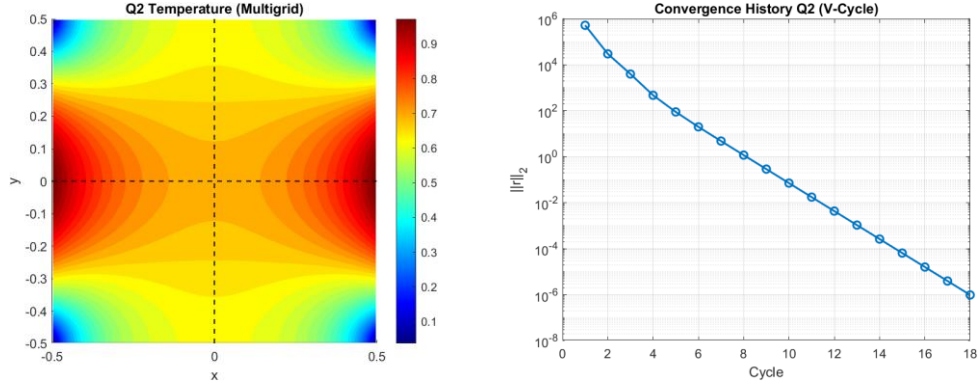


Figure 2: Q2 results

The results in

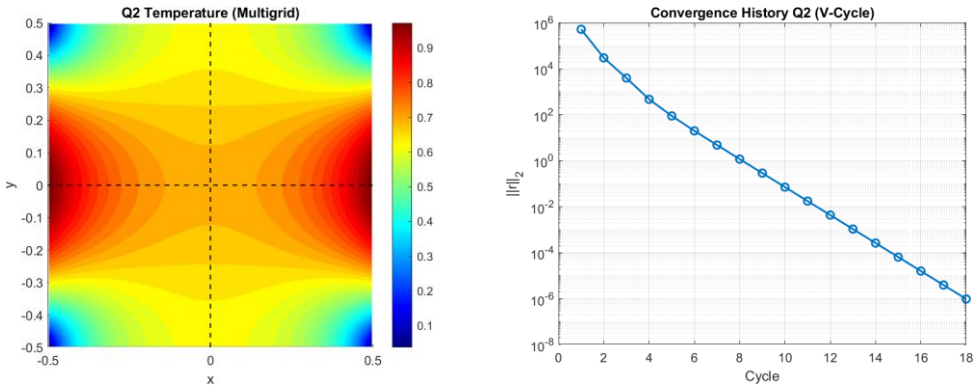


Figure 2 show that the geometric multigrid V-cycle provides a stable and efficient solution for the steady heat conduction problem with a very large conductivity contrast. The method preserved the physical effect of the sharp material interface by using harmonic face averaging together with conservative restriction and piecewise-constant prolongation, avoiding artificial smearing on coarser levels. At the same time, the combination of line Gauss-Seidel smoothing, vectorized residual evaluation, and one-time coefficient assembly per grid level significantly reduced the computational cost, allowing the solution to reach the prescribed tolerance in far fewer cycles than a single-grid iterative approach.

III. Q3

we solve the steady-state problem using the BiCGStab algorithm with a single Multigrid V-cycle integrated as a left preconditioner. The primary challenge in this stage is the 10^{-5} conductivity ratio, which results in an extremely ill-conditioned system matrix that is difficult to solve with standard iterative methods. By using the V-cycle as a preconditioner, we dampen high-frequency errors while BiCGStab provides the global stability necessary to handle the sharp material interface. We maintained the line

Gauss-Seidel smoother and harmonic averaging from the previous sections, as they are essential for accurately capturing the heat flux and managing the directional bias caused by the high-conductivity quadrant.

To meet the efficiency targets for the upcoming complexity analysis in Question 4, we implemented the entire solver using a fully vectorized architecture. A critical technical hurdle when using a preconditioner is ensuring total consistency between the global matrix A and the Multigrid operators. We addressed this by using a unified assembly routine to build both the sparse matrix and the coarse-grid hierarchy from the same physics logic, preventing the solver from stalling due to operator mismatch. For maximum speed, we utilized matrix-free residual calculations within the recursive V-cycle calls and employed the repelem function for fast grid prolongation. This optimized approach allows our system to reach the required six-decimal-digit accuracy in only a few global iterations.

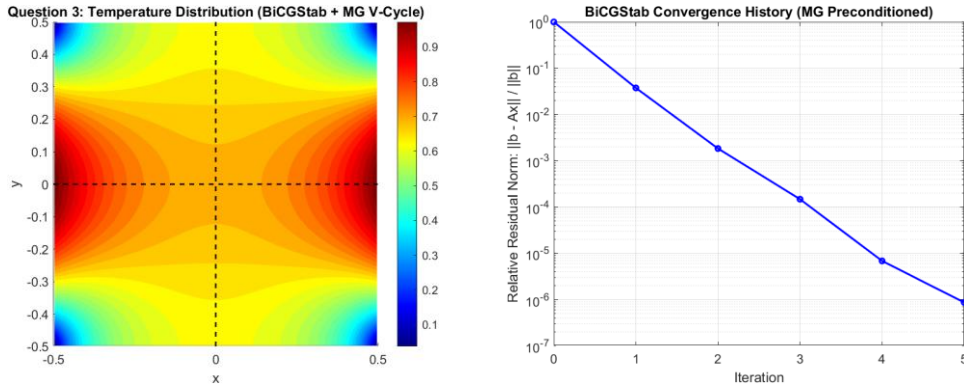


Figure 3: Q3 results

The results of Question 3 show that combining BiCGStab with a single Multigrid V-cycle as a left preconditioner provides a robust and efficient solver for the steady heat-conduction problem with strongly discontinuous conductivity. Despite the severe ill-conditioning caused by the 10^5 conductivity ratio, the preconditioned solver converged to the required six-decimal-digit accuracy in only a few global iterations, while preserving the physically correct heat-flux behavior through harmonic face averaging and the line Gauss-Seidel smoother inherited from the previous sections. The good convergence behavior confirms that the Multigrid cycle effectively removes the error components that are difficult for a Krylov method to handle alone, and that maintaining full consistency between the global sparse operator and the multigrid hierarchy is essential for stable and reliable performance.

IV. Q4

To answer Question 4, we compared the runtime of the two methods from Items 2 and 3 under the same accuracy requirement. Since the question asks for a solution accurate to six decimal digits, we did not rely only on the residual, and instead first computed a highly accurate reference solution on each grid using a very tight tolerance. We then calibrated both solvers so that they reached the same true accuracy level, defined by

$$\|T - T_{ref}\|_{\infty} < 5 \times 10^{-7}.$$

For the pure multigrid method, we determined the number of V cycles required to satisfy this condition, and for the BiCGStab solver with one multigrid V cycle as a left preconditioner, we determined the loosest residual tolerance that still produced the same accuracy. After this calibration stage, we timed both methods on a sequence of grid sizes and estimated the computational complexity from a log-log fit of the form $t(N) \sim N^{\beta}$. The main challenge was ensuring that the comparison was truly fair, since a fixed solver tolerance does not automatically guarantee six correct decimal digits in the solution, especially when comparing two different iterative methods. we addressed this by introducing a reference solution and calibrating each method against the same true error criterion. we also used powers of two for the grid sizes so that the multigrid hierarchy could coarsen consistently, repeated each timing test several times and averaged the results to reduce timing noise, and added checks during the BiCGStab timing stage to make sure that failed runs or runs that did not reach the required accuracy were not treated as valid timing results.

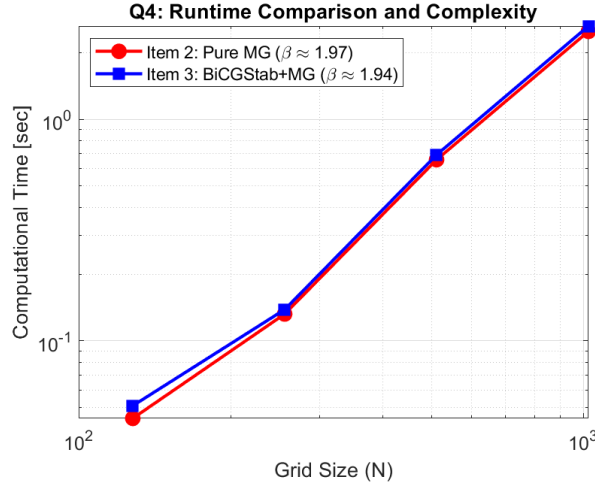


Figure 4: Q4 results

The Q4 results show that both the pure Multigrid solver and the BiCGStab solver with a single MG V-cycle preconditioner scale almost identically over the tested grid sizes. From the log-log fit in Figure 4, the measured exponents are approximately $\beta \approx 1.97$ for pure MG and $\beta \approx 1.94$ for BiCGStab+MG when the runtime is plotted versus N , which corresponds to an almost linear dependence on the total number of unknowns $n \sim N^2$. In addition, the runtime curves remain very close across all tested grids, with pure MG being slightly faster in the present implementation. Overall, these results confirm that both methods are highly efficient for the steady heterogeneous heat-conduction problem, and that the multigrid-based preconditioning strategy preserves near-optimal complexity while delivering performance comparable to the standalone multigrid solver.

V. Q5: Unsteady Heat Conduction on a Body-Fitted Triangular Mesh

A. Numerical Method

In Question 5, we solved the unsteady heat equation using a cell-centered finite volume method on the body-fitted triangular mesh generated by the provided MATLAB script. Since the mesh conforms to the circular hole, the boundary is represented directly by the grid and does not suffer from the stair-step error of a Cartesian discretization. Because the triangular mesh is generally non-orthogonal, a simple two-point flux approximation is not sufficient by itself. Therefore, the orthogonal part of the diffusion operator was assembled implicitly, while the remaining non-orthogonal contribution was treated using the Minimum Correction Method

together with Green-Gauss gradient reconstruction. In the explicit cross-diffusion term, the face gradient was interpolated from neighboring cell gradients using geometry-based weights, in the form $\nabla T_f = g_C \nabla T_C + g_F \nabla T_F$, which is consistent with the courses formulation of the minimum-correction approach. The same treatment was applied on interior faces, and the cross-diffusion contribution on Dirichlet boundaries was computed from the owner-cell gradient. The boundary conditions were imposed according to the project statement: $T = 2$ on the hole boundary, $T = 1 - 4y^2$ on the left and right walls, and zero normal heat flux on the insulated top and bottom walls. To deal with the strong conductivity jump between $k_1 = 10^{-3}$ and $k_2 = 100$, harmonic averaging was used on interior faces. Time integration was performed with the fully implicit backward Euler scheme using $\Delta t = 10^{-3}$, and the resulting sparse linear system was solved with a direct LU factorization. Since the matrix does not change in time, the LU decomposition was computed only once and then reused throughout the simulation. The non-orthogonal correction was treated in deferred form and updated through a small fixed-point loop inside each time step until the correction converged. The initial temperature field was taken as $T = 0$ in the entire plate. In addition, the geometric face data were precomputed once, and the correction terms were assembled in vectorized form in order to reduce unnecessary loops and improve efficiency

B. Results

The temperature field was recorded at the required times $t = 1$, $t = 5$, and at the steady state. At $t = 1$, the solution is still in the early transient stage, and the strongest gradients appear near the circular hole, which acts as the hot boundary. At $t = 5$, the heat has diffused further into the plate, and the influence of the side-wall temperature profile becomes more visible in the overall shape of the field. The final steady-state solution, reached at approximately $t = 142$ in the present simulation, is smooth and free of visible numerical oscillations. The contour plots show that the temperature remains within the expected range and follows the physical diffusion pattern imposed by the boundary conditions and the conductivity distribution. The corresponding 3D surface plots help visualize the thermal topography more clearly, especially the elevated temperature region around the hole and the gradual decay toward the colder boundaries. Overall, the updated results confirm that the fully implicit finite-volume formulation,

together with the minimum-correction treatment of mesh non-orthogonality, provides a stable and physically consistent solution for the problem.

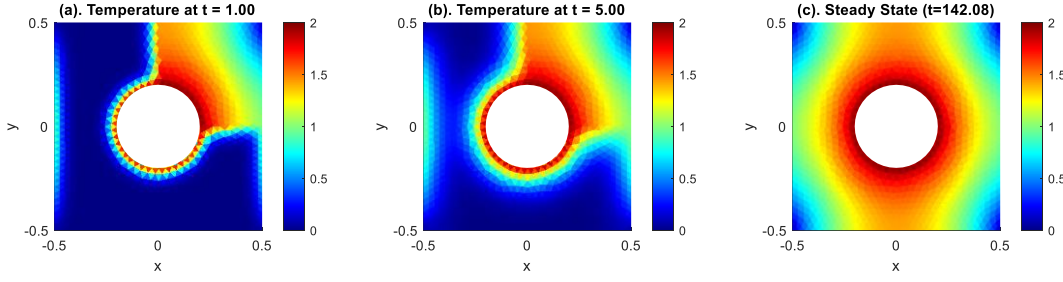


Figure 5

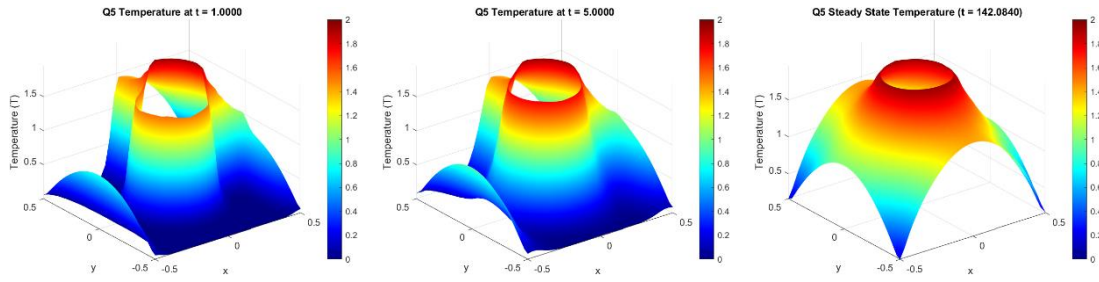


Figure 6: trisurf results using the command line

VI. Q6: Heat Diffusion in Heterogeneous Media using Immersed Boundary Method and Schur Complement

The unsteady 2-D heat conduction problem in the presence of a circular hole is solved using a fully implicit, direct-forcing Immersed Boundary Method (IBM) on a 200×200 structured Cartesian grid. To enforce the Dirichlet boundary condition ($T = 2$) on the immersed interface, a set of Lagrangian markers is distributed along the circular boundary. The coupling between the Eulerian grid and the Lagrangian points is achieved through interpolation and spreading operators utilizing a four-point regularized Peskin delta kernel. Due to the fully implicit time integration, the resulting algebraic system takes a saddle-point form, which is solved efficiently using a Schur complement formulation.

The solution procedure involves three primary steps: (1) predicting an intermediate temperature field by solving the Eulerian diffusion operator without the boundary force; (2) solving the reduced Schur system to determine the Lagrange multipliers (forcing

terms) required to satisfy the boundary condition; and (3) applying a correction step to the temperature field. To account for the discontinuous thermal conductivity distribution (k_1, k_2) , a conservative finite-volume discretization is employed with harmonic averaging of the conductivity at the cell faces. For computational efficiency, both the Eulerian operator and the Schur matrix are pre-factorized using LU decomposition outside the time loop, significantly reducing the per-step computational cost while maintaining high numerical stability.

VII. Q7: Section 7 – Numerical Methods (IBM)

This section addresses the unsteady diffusion equation around a thermally insulated hole by enforcing zero normal heat flux (Neumann BC). The solution utilizes the Explicit Direct Forcing IBM, which allows for the representation of complex geometries on a uniform Cartesian grid without the need for a body-fitted mesh.

Methodology Overview:

- **Stability:** The time step is governed by a strict CFL condition to prevent numerical divergence.
- **Predictor-Corrector:** The solution follows a two-step process: calculating a predicted temperature field (T^*) via the diffusion operator, and subsequently updating it using a Direct Forcing term.
- **Probe & Ghost Strategy:** The zero-flux condition is enforced by equating temperatures between Probe points (located in the active domain) and Ghost points (located inside the hole).
- **Efficiency:** To optimize runtime, Sparse Matrices are pre-computed during the setup phase. These matrices facilitate rapid interpolation and spreading of forces using a smoothed Delta function (Deltafunc), replacing expensive point-wise calculations in the main loop.

The final Eulerian temperature field is updated at each time step according to:

$$T^{n+1} = T - dt * Q_ibm$$

A. Pre-processing Stage

Before entering the time-stepping loop, the code performs several critical operations aimed at improving numerical stability and computational efficiency.

1. Time Step Calculation (Stability Criteria)

The time step is computed as:

$$dt = 0.9 \cdot h^2 / (4 \cdot \max(k_1, k_2))$$

This formulation ensures compliance with the CFL stability condition. The use of the maximum conductivity along with a safety factor of 0.9 prevents numerical divergence and guarantees a stable solution when using an explicit scheme.

2. Adaptive Tolerance

When the time step dt is very small, temperature changes between iterations become negligible, which may lead to premature convergence when using a fixed tolerance. To prevent this, an adaptive tolerance is defined as:

$$tolerance = dt \cdot 10^{-6}$$

This scaling ensures that the convergence criterion remains consistent with the time step size and reflects true physical convergence rather than insignificant numerical updates.

3. Diffusion Operator Assembly (AssembleLaplacian)

The function `AssembleLaplacian` constructs a sparse matrix representing the discrete Laplacian operator.

The conductivity between adjacent cells is computed using a harmonic average:

$$k_w = (2 \cdot k_i \cdot k_{i-1}) / (k_i + k_{i-1})$$

This approach ensures continuity of heat flux across interfaces between regions with different conductivities (k_1, k_2) and enforces the correct external boundary conditions.

4. IBM Operators Construction `IBMMatrix`, `Deltafunc`, `Assemble_IBM_Operators`

The function `IBMMatrix` constructs interpolation and spreading matrices that couple:

- The Eulerian grid

- The Lagrangian points located on the hole boundary

It relies on the function `Deltafunc`, which implements a smoothed delta function (based on Roma et al.).

This smoothing is essential for:

- Preventing numerical oscillations
- Enhancing stability near the immersed boundary

The function `Assemble_IBM_Operators` establishes a dual-ring Lagrangian framework by generating probe and ghost points strategically offset from the boundary. This configuration enables the precise interpolation of local field properties and the distribution of corrective forces necessary to enforce the zero-flux Neumann condition.

B. Main Algorithm: Predictor–Corrector Cycle

At each time step, a closed iterative process is performed to enforce boundary conditions on the immersed object without modifying the underlying grid.

1. Prediction Step

A temporary temperature field is computed using an explicit diffusion scheme:

$$T = T^n + dt \cdot (L \cdot T^n + b_{bc})$$

Where:

- L — discrete Laplacian operator
- b_{bc} — boundary condition contribution

2. Interpolation (Eulerian $\rightarrow$ Lagrangian)

The temperature field is interpolated from the Eulerian grid to Lagrangian sampling points:

$$\begin{aligned} T_p &= M_{probe} \cdot T \\ T_g &= M_{ghost} \cdot T \end{aligned}$$

Where:

- T_p — temperature at Probe points (outside the hole)

- T_g — temperature at Ghost points (inside the hole)

3. Forcing Calculation

The flux required to enforce a Neumann (zero-flux) boundary condition is computed as:

$$Q_{lag} = -(T_p - T_g) / (d_{total})$$

$$Q_{lag} = Q_{lag}/h$$

This quantity represents the correction required at each Lagrangian boundary point, with dimensional scaling, in order to match source term units.

4. Spreading (Lagrangian → Eulerian)

The computed flux is distributed back to the Eulerian grid using a spreading operator:

$$Q_{ibm} = M_{spread} \cdot (Q_{lag} \cdot K_{lag})$$

This step represents a discrete integral, translating the boundary effect into a source term in the global diffusion equation.

5. Final Correction Step

The temperature field is updated as follows:

$$T^{n+1} = T - dt \cdot Q_{ibm}$$

The resulting temperature field satisfies both the diffusion equation and the imposed boundary condition on the immersed boundary.

C. Optimization: Delta Function Pre-computing

To improve computational efficiency, the delta function is not evaluated inside the main time loop.

Pre-processing Stage

The function `IBMatrix` computes all interpolation and spreading weights once before the simulation begins and stores them in sparse matrices.

Weight Matrix

Each entry in the matrix: $M(i,j)$ represents the value of the delta function between Eulerian cell j and Lagrangian point i . The matrix serves as a fixed representation of the geometry, eliminating the need for repeated evaluations.

Usage in the Main Loop

Interpolation is performed by matrix-vector multiplication:

$$T_p = M_{probe} \cdot T$$

This operation is mathematically equivalent to a discrete convolution (integral), but significantly more efficient computationally.

Note: The use of pre-computing does not introduce any additional numerical approximation. It is purely a computational optimization, replacing repeated evaluations with stored values, without compromising solution accuracy.